

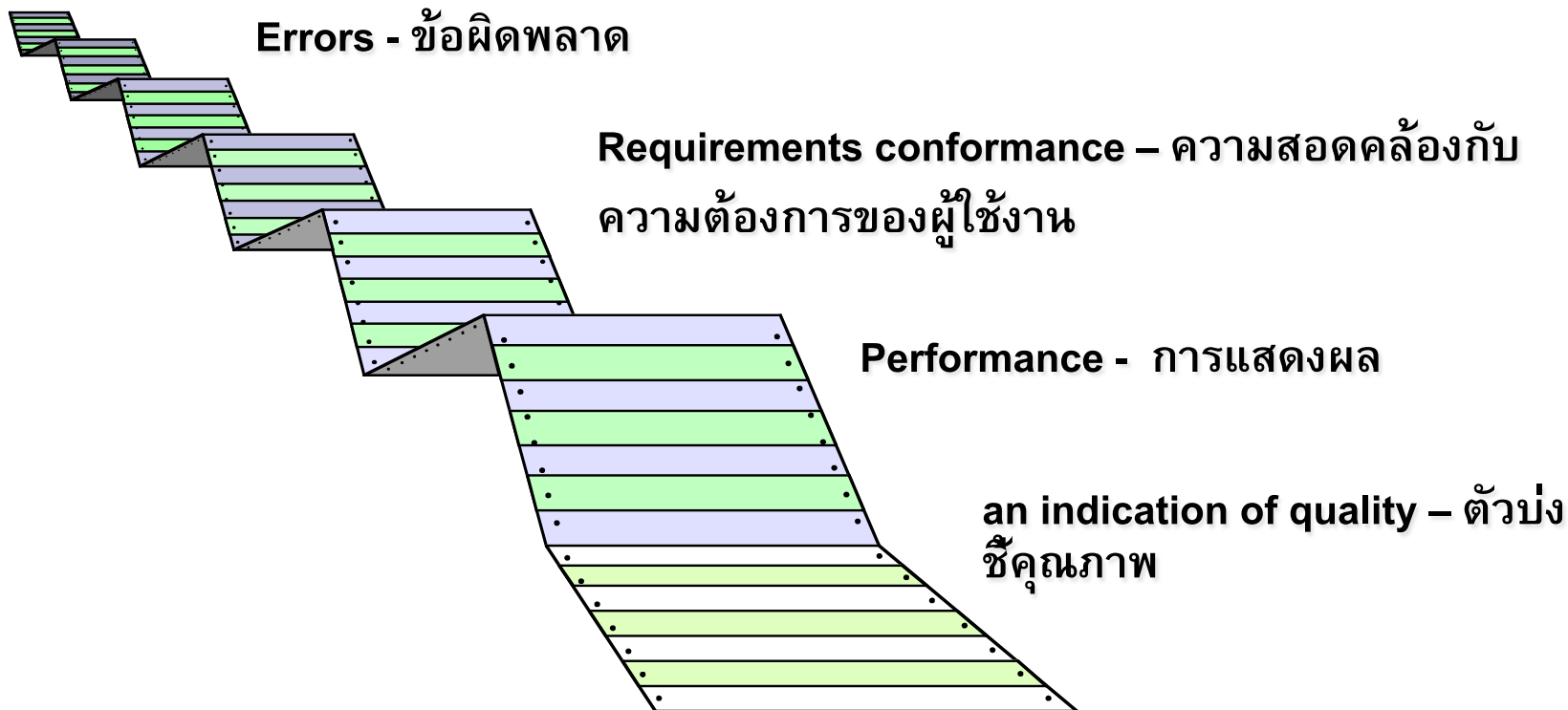
วิศวกรรมซอฟต์แวร์ (Software Engineering)

□ กลยุทธ์การทดสอบซอฟต์แวร์ (Testing Strategies)

กลยุทธ์การทดสอบซอฟต์แวร์

- ❑ กลยุทธ์การทดสอบซอฟต์แวร์ – วิธีการออกแบบกรณีทดสอบและการวางแผนทดสอบ เพื่อให้ได้ชุดของขั้นตอนตามที่ปฏิบัติ เป็นการยืนยันว่าการสร้างซอฟต์แวร์ประสบความสำเร็จ
- ❑ กลยุทธ์ใดๆ ต้องมีแผนการทดสอบ การออกแบบกรณีทดสอบ การลงมือทดสอบ และการรวบรวมและประเมินผลข้อมูลผลลัพธ์

What Testing Shows



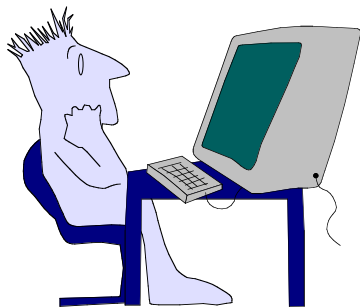
แนวทางการทดสอบซอฟต์แวร์เชิงยุทธวิธี

กลยุทธ์การทดสอบซอฟต์แวร์ มีลักษณะทั่วไปดังนี้

- ❑ การปฏิบัติการทดสอบให้ได้ผล ควรตรวจทานเอกสารทางเทคนิค เพื่อลดข้อผิดพลาดก่อนเริ่มทดสอบจริง
- ❑ การทดสอบเริ่มที่องค์ประกอบย่อยก่อนแล้วเป็นภาพรวมของระบบทั้งหมด
- ❑ เทคนิคที่แตกต่างกันเหมาะสมกับการทดสอบในจุดต่าง ๆ กันตามกาลเวลา
- ❑ การทดสอบ กระทำโดย ผู้พัฒนาซอฟต์แวร์และนักทดสอบอิสระ
- ❑ การทดสอบและการดีบั๊กเป็นกิจกรรมที่ต่างกัน แต่การดีบั๊กเป็นสิ่งที่เกิดขึ้นควบคู่ไปกับการทดสอบ

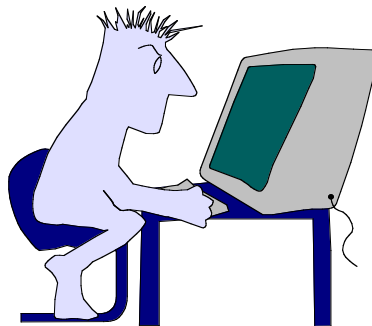
ผู้ทดสอบซอฟต์แวร์

developer



**Understands the system
but, will test "gently"
and, is driven by "delivery"**

independent tester



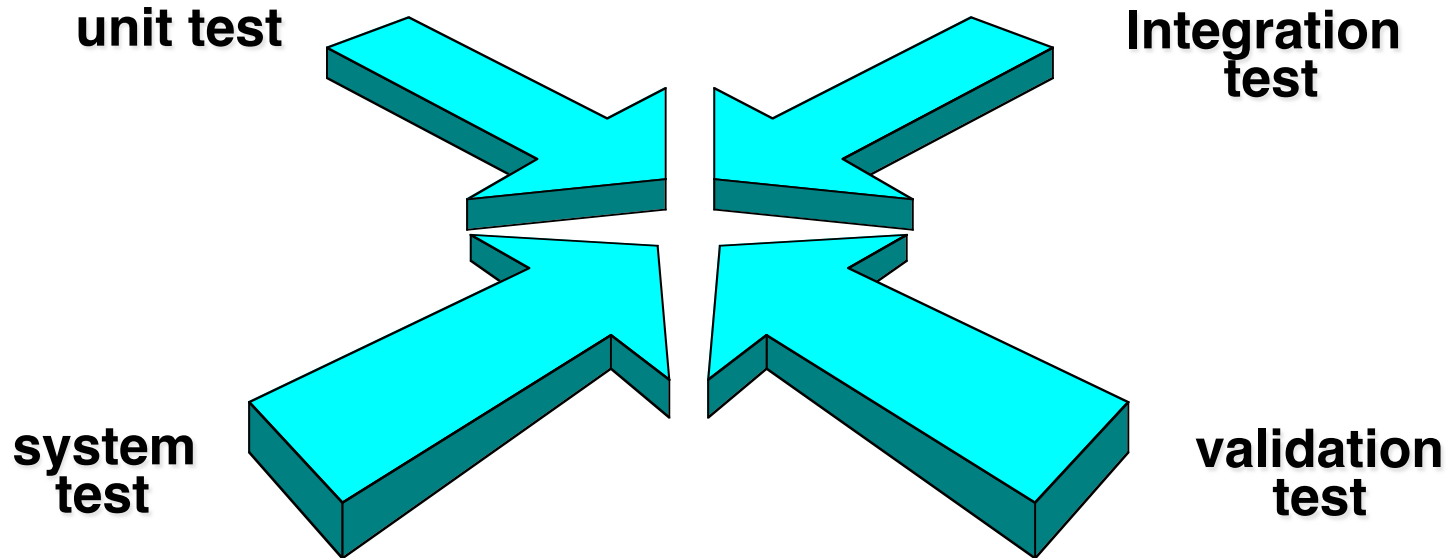
**Must learn about the system,
but, will attempt to break it
and, is driven by quality**

Verification & Validation

- ❑ Verification – ชุดของกิจกรรมที่ประกันว่า ซอฟต์แวร์ได้ทำตามหน้าที่เฉพาะที่กำหนดขึ้น (ได้สร้างผลิตภัณฑ์อย่างถูกต้องหรือไม่?)
- ❑ Validation – ชุดของกิจกรรมที่ต่างออกไปที่ประกันว่าซอฟต์แวร์ที่สร้างขึ้นเป็นไปตามความต้องการของลูกค้า (ได้สร้างผลิตภัณฑ์ที่ถูกต้องหรือไม่?)
- ❑ V&V เป็นกิจกรรม การประกันคุณภาพของซอฟต์แวร์ (Software Quality Assurance – SQA)

Testing Strategy

We begin by 'testing-in-the-small' and move toward 'testing-in-the-large'



Strategic Issues

- ❑ State testing objectives explicitly. - ระบุวัตถุประสงค์การทดสอบอย่างชัดเจน
- ❑ Understand the users of the software and develop a profile for each user category. – เข้าใจผู้ใช้งาน และพัฒนารูปแบบการใช้งานสำหรับผู้ใช้แต่ละกลุ่ม
- ❑ Develop a testing plan that emphasizes “rapid cycle testing.” – สร้างแผนการทดสอบที่เน้น การทดสอบในวงรอบเร็วสูง
- ❑ Build “robust” software that is designed to test itself – สร้างซอฟต์แวร์ที่ทนทานและออกแบบมาให้ทดสอบตัวเองได้

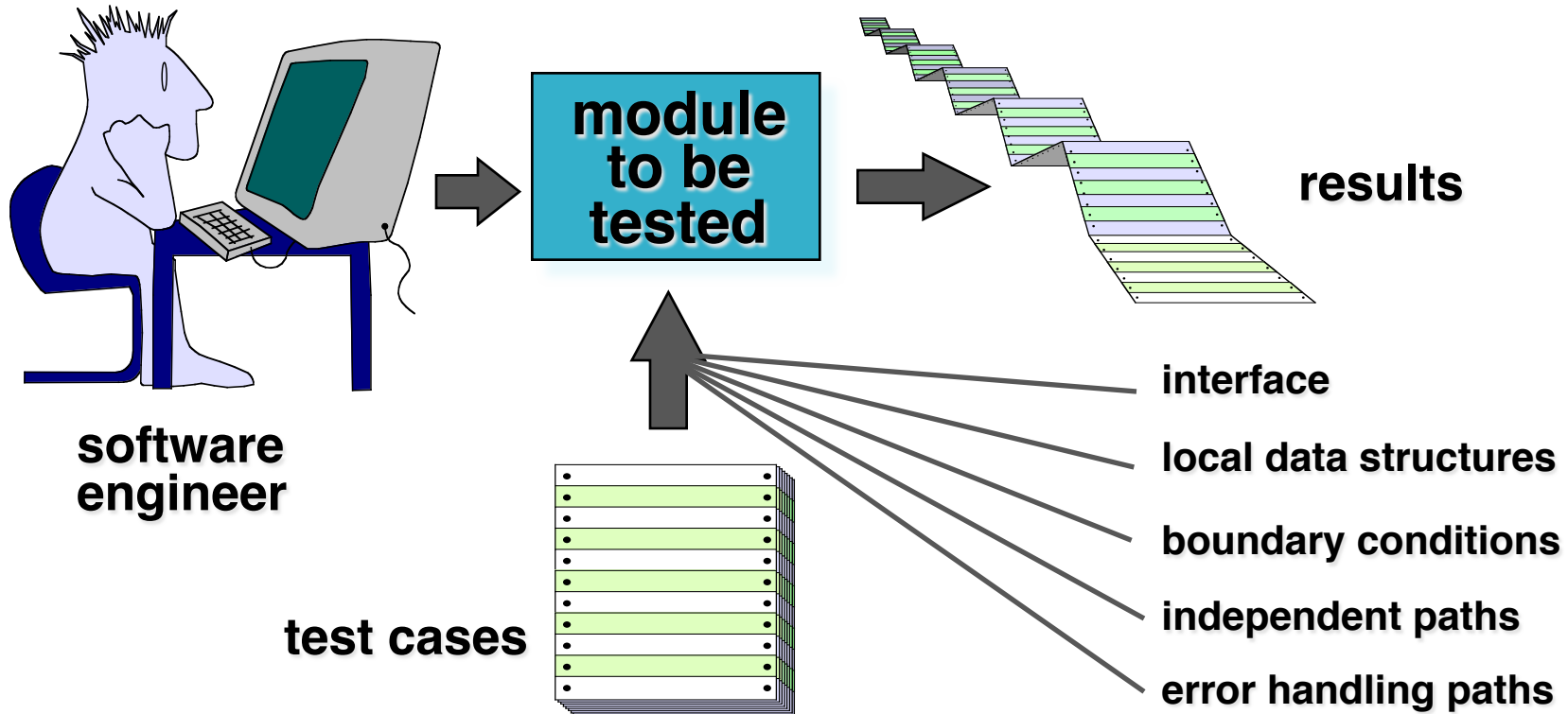
Strategic Issues

- ❑ Use effective formal technical reviews as a filter prior to testing - ใช้การตรวจสอบอย่างเป็นทางการเป็นตัวกรองก่อนการทดสอบ
- ❑ Conduct formal technical reviews to assess the test strategy and test cases themselves. - ทำการตรวจสอบด้านเทคนิคอย่างเป็นทางการเพื่อประเมินกลยุทธ์การทดสอบที่จะใช้
- ❑ Develop a continuous improvement approach for the testing process. - พัฒนาแนวทางปรับปรุงกระบวนการทดสอบอย่างต่อเนื่อง

Unit Testing

- การทดสอบระดับหน่วย เจาะจงทดสอบหน่วยที่เล็กที่สุดของงานออกแบบซอฟต์แวร์ คือ องค์ประกอบซอฟต์แวร์ (Component) หรือโมดูลซอฟต์แวร์ (Module) เพื่อประเมินการทำงานในด้านต่างๆ ของหน่วยย่อย ไม่ว่าจะเป็นด้านโครงสร้างข้อมูล โครงสร้างควบคุมการทำงาน ตลอดจนการรองรับการทำงานเมื่อเกิดความผิดพลาด

Unit Testing



Unit Testing

- ❑ **Interface** – การทดสอบข้อมูลผ่านโมดูลอินเตอร์เฟส ว่าข้อมูลเข้าและออก ถูกต้องหรือไม่
- ❑ **Local data structures** – โครงสร้างข้อมูลระดับหน่วยที่มีขอบเขตอยู่ภายในหน่วยย่อย เพื่อทดสอบดูว่าข้อมูลที่ถูกจัดเก็บไว้ชั่วคราวมีความสมบูรณ์หรือถูกต้องระหว่างการทำงานของอัลกอริทึมหรือไม่
- ❑ **Boundary conditions** – เงื่อนไขขอบเขต คือขอบเขตค่าข้อมูลที่โปรแกรมต้องประมวลผล โปรแกรมจะต้องทำงานภายใต้ขอบเขตข้อมูลที่กำหนดจึงจะถูกต้อง

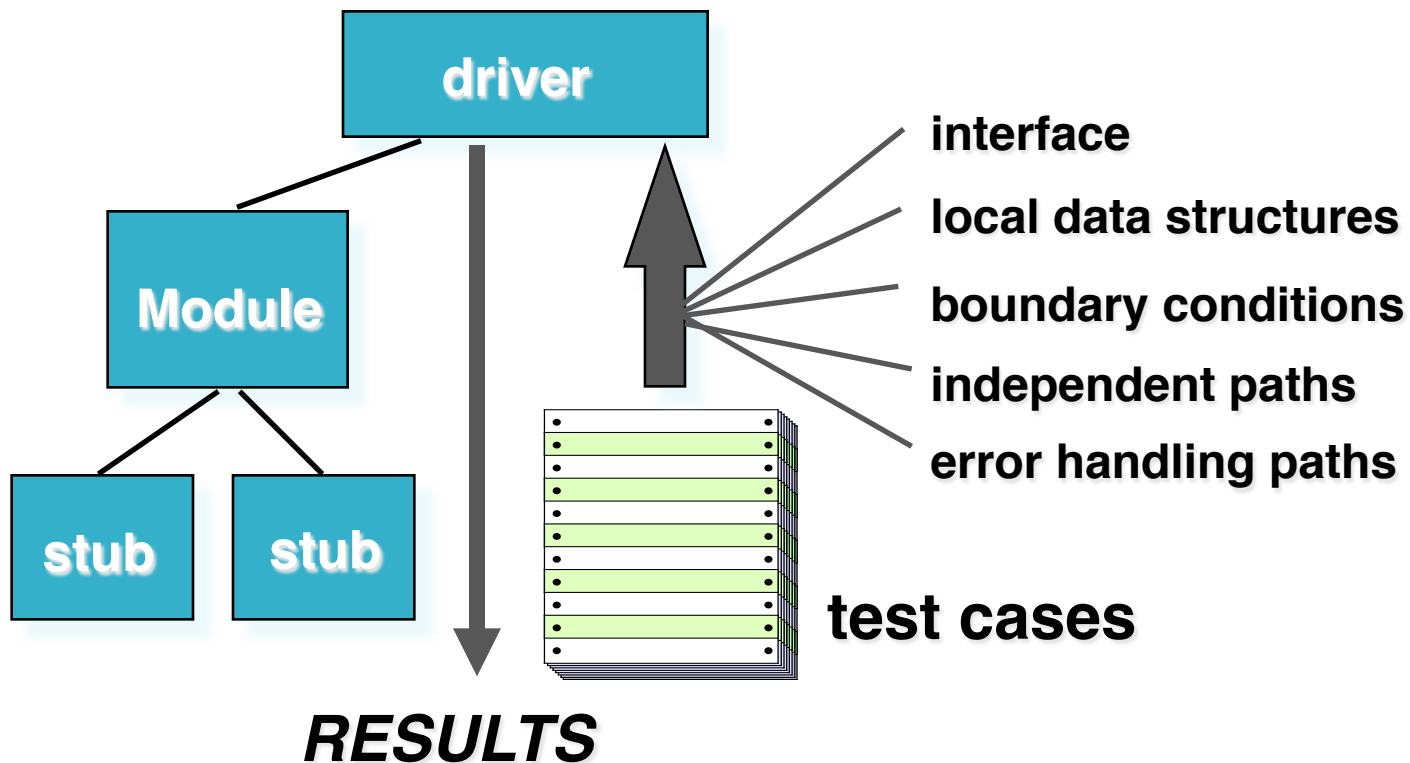
Unit Testing

- ❑ **Independent paths** – เส้นทางการทำงานที่แตกต่างกันตามเงื่อนไขที่ได้รับ เมื่อนำข้อมูลเข้าสู่โปรแกรมจะต้องมีเส้นทางอย่างน้อย 1 เส้นทางที่จะถูกเรียกใช้งาน
- ❑ **Error handling paths** – เส้นทางการประมวลผลข้อผิดพลาดและแสดงข้อผิดพลาด เมื่อมีข้อผิดพลาดเกิดขึ้น จะต้องส่งไปยังเส้นทางการประมวลผลข้อผิดพลาด เพื่อแจ้งเตือนผู้ใช้และคำแนะนำเพื่อทำงานต่อไป

Unit Test Environment

- เนื่องจาก Component ไม่ใช้ระบบที่สมบูรณ์ ดังนั้น Driver หรือ Stub จึงจำเป็นสำหรับการทดสอบระดับหน่วย
- Driver เป็นโปรแกรมหลักที่รับข้อมูลกรณีทดสอบ แล้วส่งผ่านข้อมูลไปยังองค์ประกอบที่ถูกทดสอบคือ stub
- Stub คือโปรแกรมดัมมี่ (Dummy Program) ที่มีอินเตอร์เฟซหน้าต่างาเดียวกับโมดูลจริงที่ถูกเรียกใช้งาน แต่มักไม่มีการประมวลผลข้อมูล มีเพียงการตรวจทานความถูกต้องของการควบคุมข้อมูลเข้าและออกของโมดูลที่ทำการทดสอบอยู่

Unit Test Environment



Integration test

- เป็นการทดสอบการทำงานของกลุ่มโปรแกรมหรือส่วนประกอบย่อยที่ถูกประสานเข้าด้วยกัน โดยทำงานหน้าที่ใดหน้าที่หนึ่งร่วมกัน เพื่อค้นหาข้อผิดพลาดที่อาจเกิดขึ้นได้
- การทดสอบระดับรวมหน่วย ทำได้ 2 ลักษณะ คือ
 - ▣ the “big bang” approach
 - ▣ an incremental construction strategy

Integration test

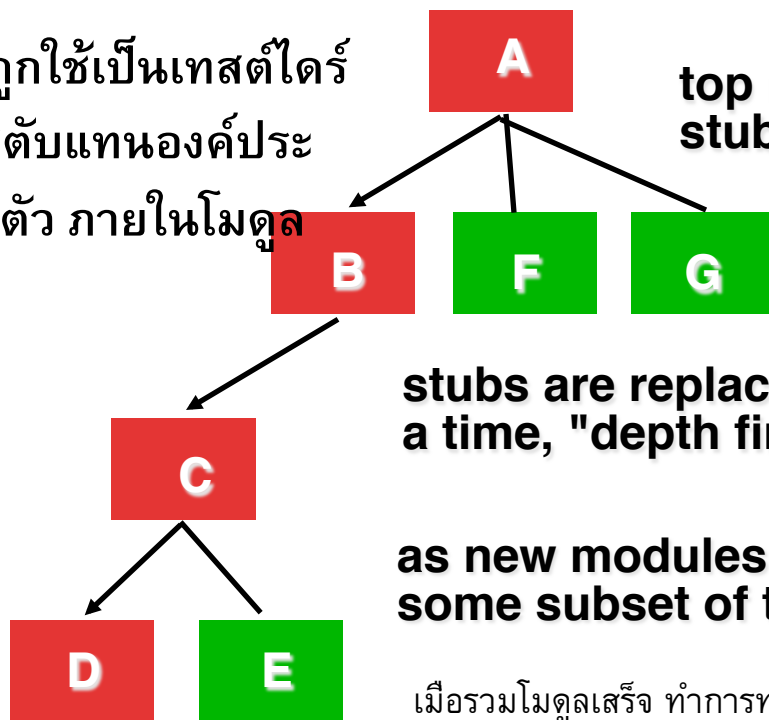
- ❑ Big bang – เป็นการนำเอาส่วนประกอบทั้งหมดมาเชื่อมกันในครั้งเดียว แล้วทดสอบโปรแกรมทั้งหมด ซึ่งเป็นการทำที่ไม่ถูกต้อง เพราะจะเกิดความยุ่งยากในการแยกหาสาเหตุและแก้ไขระบบ
- ❑ Incremental – เป็นการสร้างและทดสอบโปรแกรมจะทำจากส่วนเล็ก ๆ ก่อน แล้วค่อย ๆ เพิ่มขึ้น เมื่อพบข้อผิดพลาดก็จะง่ายในการแยกหาสาเหตุและแก้ไข

Top Down Integration

- โมดูลจะรวมเข้ากันตามลำดับขอการควบคุมจากบนลงล่าง เริ่มจากโมดูลหลัก โดยโมดูลย่อยของโมดูลหลักจะรวมเข้ากับโมดูลในเชิงลึกก่อน หรือรวมเชิงกว้างก่อนก็ได้

Top Down Integration

โมดูลหลักจะถูกใช้เป็นเทสต์ไดรเวอร์ และใช้สับแทนองค์ประกอบอื่น ๆ ทุกตัว ภายในโมดูลหลัก



top module is tested with stubs

stubs are replaced one at a time, "depth first"

แทนที่สับทีละตัวด้วยองค์ประกอบจริง ตามเชิงลึก หรือเชิงกว้าง

as new modules are integrated, some subset of tests is re-run

เมื่อรวมโมดูลเสร็จ ทำการทดสอบ และทำการรวมโมดูลย่อยอื่นต่อไป

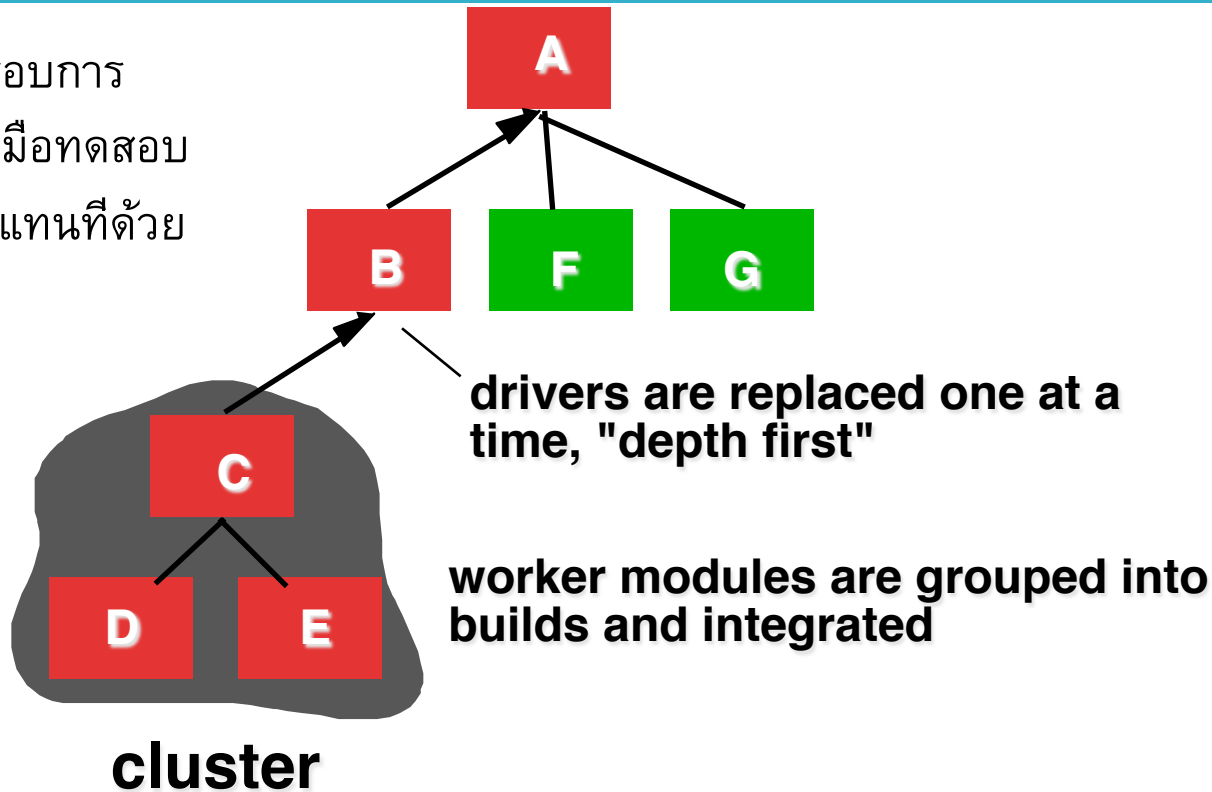
Bottom-Up Integration

- จะเริ่มการทดสอบจากโมดูลเดี่ยวๆ คือองค์ประกอบที่อยู่ระดับล่างสุดของโครงสร้างโปรแกรม เพราะรวมตัวกันจากล่างขึ้นมา การทำงานขณะรวมกันจึงเรียกใช้ขณะรวมกันในระดับนั้นๆ และไม่มีคามจำเป็นต้องใช้สลับ

Bottom-Up Integration

สร้าง Driver ขึ้นมาเพื่อทดสอบการทำงาน
ของโมดูลระดับล่าง เมื่อทดสอบ
เสร็จก็ถอด Driver ออกแล้วแทนที่ด้วย
คลัสเตอร์ใหม่

รวมโมดูลระดับล่างเข้าด้วยกัน
เป็น คลัสเตอร์ (Cluster) เพื่อ
ทดสอบการทำงานร่วมกัน



Regression Testing

- ทุกๆ ครั้งที่มีโมดูลใหม่เพิ่มขึ้นมา หมายถึงซอฟต์แวร์ได้เปลี่ยนไปแล้ว การเปลี่ยนแปลงอาจก่อให้เกิดปัญหากับหน้าที่การทำงานได้
- การทดสอบเชิงถดถอย (Regression testing) คือ การทดสอบซ้ำโดยใช้ชุดทดสอบเดิมที่เคยทำมาแล้ว เพื่อให้มั่นใจว่าการเปลี่ยนแปลงที่เกิดขึ้น ไม่ทำให้เกิดผลข้างเคียงที่ไม่น่าปรารถนา

Smoke Testing

- ❑ A common approach for creating “daily builds” for product software
การทดสอบผลิตภัณฑ์ทั้งก่อนบ่อยๆ เป็นประจำวัน
- ❑ กิจกรรมการทดสอบ
 - ▣ องค์ประกอบของซอฟต์แวร์ที่ถูกแปลงโค้ดแล้ว จะถูกรวมกันเป็น บั๊วด์ (Build) ประกอบด้วยไฟล์ข้อมูล ไลบรารี โมดูลที่ใช้ซ้ำและองค์ประกอบอื่นๆ ที่จำเป็น
 - ▣ ออกแบบชุดการทดสอบอย่างต่อเนื่องที่จะแสดงความผิดพลาดที่อาจทำให้บั๊วด์ไม่ทำงานตามปกติ
 - ▣ รวมบั๊วด์เข้ากับบั๊วด์อื่นๆ และเข้ากับผลิตภัณฑ์รวม เพื่อทำการทดสอบแบบสโมคเป็นประจำวัน

Smoke Testing

ประโยชน์ของการทดสอบแบบสโมค

- ❑ ลดความเสี่ยงเชิงบูรณาการ เพราะการทดสอบประจำวันจะพบข้อผิดพลาดเร็ว จึงลดความเสี่ยงของการล่าช้า
- ❑ ผลิตภัณฑ์สุดท้ายมีคุณภาพดีขึ้น
- ❑ ช่วยให้หาสาเหตุและการแก้ไขข้อผิดพลาดทำได้ง่าย ข้อผิดพลาดที่พบจะสัมพันธ์กับส่วนซอฟต์แวร์ใหม่ที่เพิ่มมา
- ❑ การประเมินความก้าวหน้าทำได้ง่าย เพราะแต่ละวันซอฟต์แวร์จะถูกนำมารวมกันได้มากขึ้น และเป็นตัวบ่งชี้ว่างานมีความก้าวหน้า

เอกสารกำกับการทดสอบ Integration

- ❑ Test Specification เป็นเอกสารที่บรรจุแผนการทดสอบ ขั้นตอนการทดสอบ
- ❑ แผนการทดสอบ (Test plan) เป็นเอกสารที่ประกอบไปด้วยชุดข้อมูลนำเข้าของแต่ละเส้นทางการทำงานของทุกโปรแกรม และผลลัพธ์ของการทดสอบนั้น แผนการทดสอบจะอธิบายกลยุทธ์โดยรวมของการรวมโปรแกรม
- ❑ การทดสอบแบ่งออกเป็นระยะ แต่ละระยะแสดงให้เห็นถึงการทำงานอย่างกว้างๆ ในซอฟต์แวร์ ดังนั้นบิวต์ของโปรแกรมควรถูกสร้างตามระยะเหล่านี้

เอกสารกำกับการทดสอบ Integration

- ▣ การทดสอบความน่าเชื่อถือได้ของอินเทอร์เฟซ (Interface Integrity)
 - ▣ การทดสอบความถูกต้องของหน้าที่การทำงาน (Functional Validity)
 - ▣ เนื้อหาของข่าวสาร (Information Content)
 - ▣ เกณฑ์การประกอบการ (Performance)
- ประวัติการทดสอบจริง ปัญหา ข้อน่าสงสัย ควรบันทึกในรายงานการทดสอบ (Test Report)

High Order Testing

- Validation testing

- ▣ Focus is on software requirements

- System testing

- ▣ Focus is on system integration

- ▣ เป็นการทดสอบการทำงานของระบบเมื่อรวมเข้ากับองค์ประกอบอื่นๆ ได้แก่ บุคลากร อุปกรณ์ ข้อมูล เพื่อทดสอบว่าทำงานได้ถูกต้องตามข้อกำหนดและความต้องการของผู้ใช้หรือไม่

High Order Testing

□ Alpha/Beta testing

▣ Focus is on customer usage

เป็นการทดสอบระบบโดยผู้ใช้งาน โดย alpha testing ผู้ใช้จะทดสอบระบบขณะยังไม่ได้ติดตั้งในสถานที่จริง โดยทดสอบภายในสถานการณ์จำลองที่กำหนดขึ้นโดยทีมงานพัฒนาระบบ ทีมงานจะบันทึกข้อผิดพลาดและทำการแก้ไข

beta testing เป็นการทดสอบหลังจากทดสอบแบบอัลฟาเรียบร้อยแล้ว สภาพแวดล้อมจะเป็นสภาพแวดล้อมจริง ผู้ใช้จะบันทึกข้อผิดพลาด และส่งบันทึกให้ทีมงานเป็นระยะๆ เพื่อแก้ไขข้อผิดพลาด

□ Recovery testing

- ▣ เป็นการทดสอบความสามารถในการกู้คืนระบบได้เมื่อเกิดความล้มเหลว โดยระบบจะต้องสามารถทำงานต่อไปได้และต้องทนต่อการผิดพลาดได้ คือเมื่อเกิดความล้มเหลวในส่วนใดส่วนหนึ่งจะต้องไม่ส่งกระทบให้หยุดการทำงานไปทั้งระบบ

□ Security testing

- ▣ เป็นการทดสอบการรักษาความปลอดภัยของระบบ ว่ามีเครื่องรักษาความปลอดภัยที่มีประสิทธิภาพเป็นที่น่าพึงพอใจหรือไม่ จากสถานการณ์การลักลอบเรียกใช้ข้อมูลและสถานการณ์อื่นๆ ที่เป็นภัยต่อข้อมูล

High Order Testing

□ Stress testing

- ▣ เป็นการทดสอบในสถานการณ์ไม่ปกติของระบบ เพื่อดูว่าระบบจะทนทานต่อสถานการณ์ดังกล่าวได้นานเพียงใดก่อนที่ระบบจะล้มเหลว เช่น การใช้งานมากเกินไป การนำเข้าข้อมูลปริมาณมาก ทำให้ระบบต้องใช้ทรัพยากรต่างๆ มากเช่น หน่วยความจำ เนื้อที่จัดเก็บข้อมูล

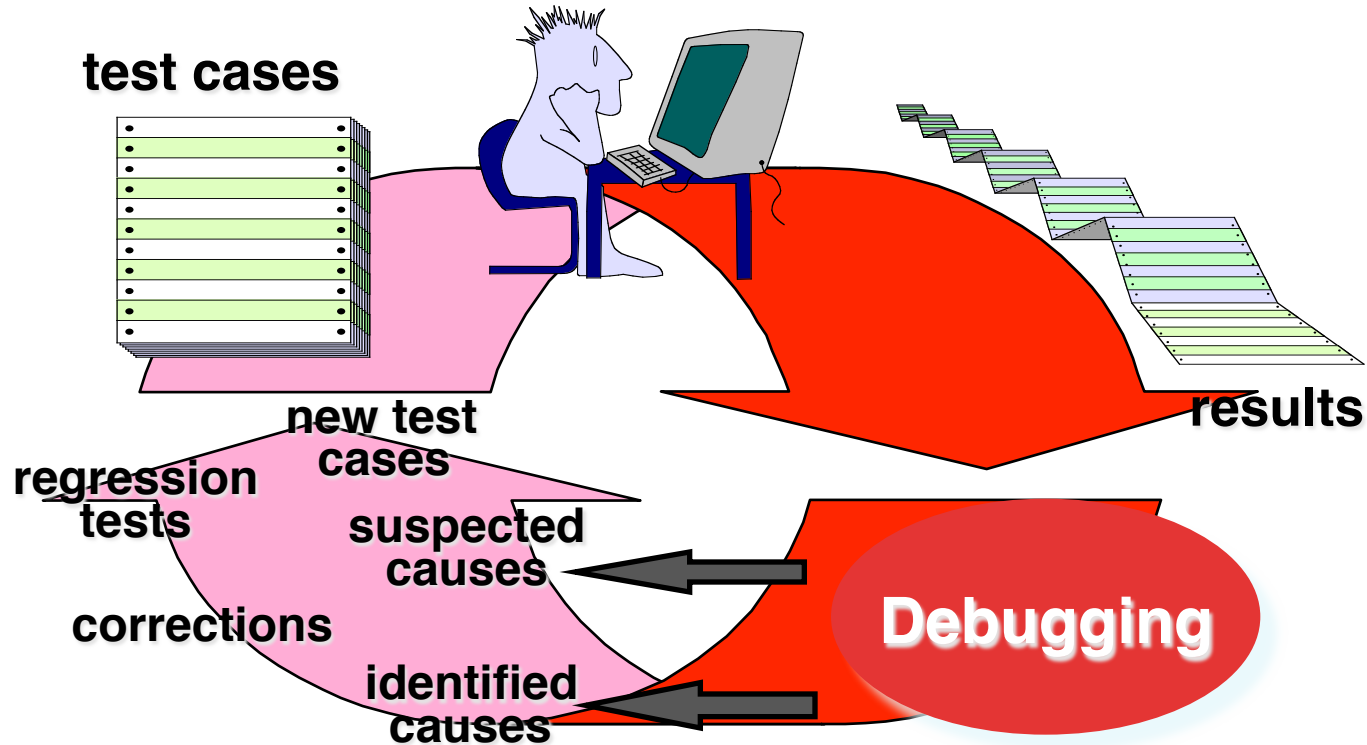
□ Performance Testing

- ▣ ทดสอบสมรรถนะด้านต่างๆ ของระบบในสถานการณ์ทั่วไปว่าอยู่ในระดับที่รับได้หรือไม่ เช่น ระยะเวลาในการตอบสนองการทำงาน การจัดสรรเนื้อที่จัดเก็บข้อมูล การจัดสรรหน่วยความจำ

การค้นหาสาเหตุของจุดบกพร่อง (Debugging)

- ❑ เมื่อกรณีทดสอบค้นพบข้อผิดพลาด การค้นหาสาเหตุของจุดบกพร่อง เป็นการกระทำที่จะแก้ไขข้อผิดพลาดที่ค้นพบ
- ❑ กระบวนการค้นหาสาเหตุของจุดบกพร่อง เป็นกระบวนการทำงานแบบวนซ้ำ เริ่มจากการทดสอบระบบ หากผลลัพธ์จริงที่ได้ไม่สอดคล้องกับผลลัพธ์ที่คาดหวัง นั้นหมายถึงมีข้อผิดพลาดซ่อนอยู่ การค้นหาสาเหตุของจุดบกพร่อง จะจับคู่ของสาเหตุและอาการซึ่งนำไปสู่การแก้ไขข้อผิดพลาด

The Debugging Process



The Debugging Process

การค้นหาสาเหตุของจุดบกพร่องมีผลสัมฤทธิ์ 2 ประการ

- สาเหตุของปัญหาถูกค้นพบ และได้รับการแก้ไข และทดสอบซ้ำ
- สาเหตุของปัญหาไม่ถูกค้นพบ ผู้แก้ไขอาจต้องเดาสาเหตุ และออกแบบทดสอบเพิ่มเติมเพื่อยืนยันการคาดเดา เพื่อนำไปสู่การแก้ไขข้อผิดพลาด

เทคนิคการค้นหาสาเหตุของจุดบกพร่อง

- ❑ Brute force -- มีประสิทธิภาพต่ำสุด ติดตามกระแสนการทำงานจริง และเพิ่มเอาต์พุตในโปรแกรม เพื่อหาร่องรอยข้อผิดพลาด
- ❑ Backtracking – เริ่มต้นจากจุดที่เกิดอาการ จากนั้นติดตามซอร์สโค้ดย้อนรอยไปถึงจุดที่เป็นสาเหตุ เหมาะสำหรับโปรแกรมขนาดเล็ก
- ❑ Cause Elimination – กำจัดทีละสาเหตุ สมมติฐานหาสาเหตุ และทดสอบแต่ละสาเหตุ กำจัดสาเหตุที่ไม่ใช่ออก ถ้าการทดสอบชี้ว่าสมมติฐานอาจเป็นสาเหตุจริง ปรับข้อมูลทดสอบให้ละเอียดขึ้น เพื่อพยายามแยกปัญหาออกมา

การแก้ไขข้อผิดพลาด

- ❑ เมื่อใดที่บั๊กถูกค้นพบแล้ว ก็ต้องได้รับการแก้ไขด้วย การแก้ไขอาจนำไปสู่ข้อผิดพลาดอื่นๆ ได้อีก โดยอาจทำให้เกิดความเสียหายมากกว่าเดิม ก่อนลงมือแก้ไขให้ตอบคำถามต่อไปนี้
 - ❑ สาเหตุของบั๊กนี้เกิดได้ในส่วนอื่นๆ ของโปรแกรมหรือไม่
 - ❑ ก่อนที่จะแก้ไข พิจารณาวามีบั๊กอื่นอะไรอีกบ้างที่อาจจะเกิดขึ้นการแก้บั๊กนี้
 - ❑ จะทำอะไรได้บ้างที่จะป้องกันความผิดพลาดนี้